PNY Trainings — Cyber Security Department

Web Application Security — Final Project Assignments

Project 3: Broken Access Control Audit

Forced Browsing & Insecure Direct Object Reference (IDOR)
Findings Report — OWASP Juice Shop

Student: Hamza Iqbal

Environment: Local lab — Docker on Windows 11 (own host)

Table of Contents

1. Objective

2. Scope & Rules of Engagement

3. Tools & Test Accounts

4. Test 1 — Forced Browsing to Administrative Functions

5. Test 2 — Finding 2: Insecure Direct Object Reference (Basket IDOR)

6. Findings Summary

7. Annex A — Reusable Broken Access Control Checklist

8. Conclusion

1. Objective

The objective of this project was to audit the target application's access control: to test whether functions and data restricted to administrators or to other users can be reached by an ordinary authenticated user, by (a) forced browsing directly to administrative routes and APIs, and (b) tampering with object identifiers (IDOR) to reach another user's data. Each confirmed weakness is documented with a severity rating, evidence, and a fix, and the tests are generalized into a reusable audit checklist (Annex A).

2. Scope & Rules of Engagement

- **Target —** OWASP Juice Shop v20.3.0, deployed locally via Docker, at http://localhost:3000
- **Environment —** Isolated lab on the tester's own Windows 11 host; all accounts used were created for this audit.
- **Boundary —** No external system was touched, consistent with the lab-only scope of the assignment.

3. Tools & Test Accounts

Tool	Version	Purpose
Docker Desktop	4.95.0	Container runtime for the target application
OWASP Juice Shop	20.3.0 (bkimminich/juice-shop)	Target application under test
Google Chrome + DevTools	155	Forced browsing, session-storage inspection and tampering, evidence capture

Account	Role	Basket ID	Purpose
attacker@test.com	Normal user	6	Attacker — performs forced browsing and ID tampering
victim@test.commm	Normal user	7	Victim — owns the basket targeted by the IDOR test (2 × Apple Juice)

The two accounts were used in separate browser contexts (a normal window and an incognito window) so their sessions and baskets stayed independent.

4. Test 1 — Forced Browsing to Administrative Functions

Hypothesis: pages and APIs intended for administrators might be reachable by a normal user who requests their URLs directly ("forced browsing"), if authorization is enforced only by hiding links in the user interface. Two targets were tested while authenticated as attacker@test.com.

4.1 Administration page (/#/administration) — control effective

Step 1: Authenticate as the normal user attacker@test.com.

Step 2: Enter http://localhost:3000/#/administration directly in the address bar.

Step 3: Result: the application responds with a 403 page — "You are not allowed to access this page!" (Figure 1).

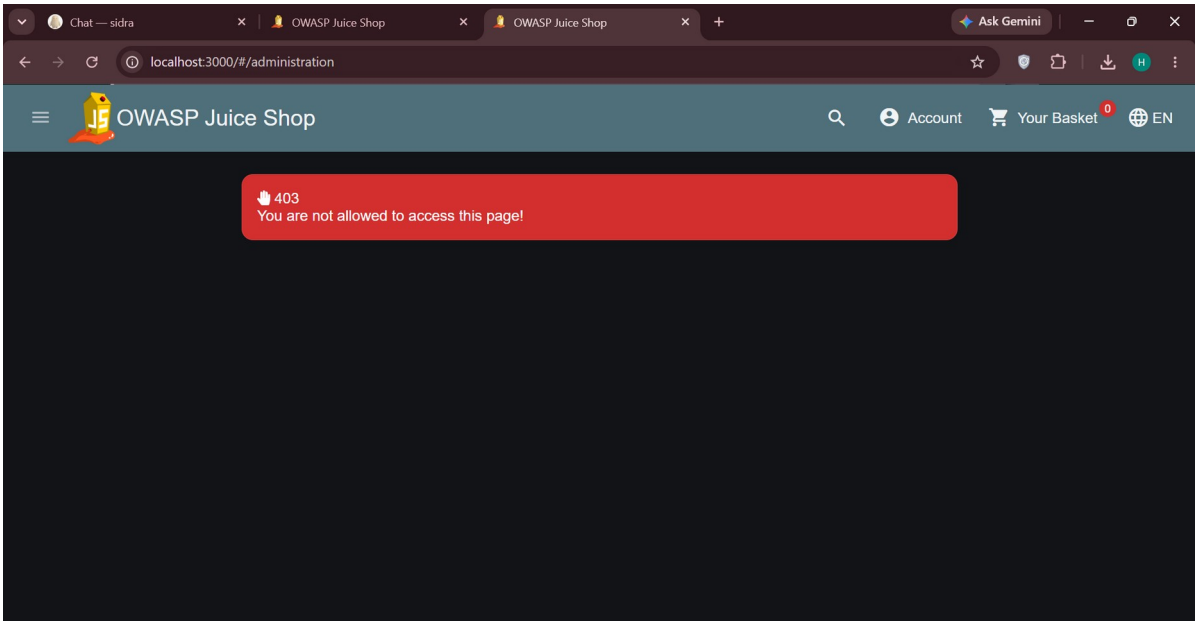


Figure 1 — The administration page correctly rejects the non-admin user with a 403

Assessment: for the web page itself, access control works as intended. This negative result is recorded deliberately — an audit documents controls that pass as well as those that fail. The next test checks whether the backend API behind the admin area is enforced to the same standard.

4.2 Finding 1 — Administrative configuration API exposed to any user

Severity: High

Affected endpoint: GET /rest/admin/application-configuration

CWE: CWE-862 — Missing Authorization

Step 1: In a new tab, request http://localhost:3000/rest/admin/application-configuration while still authenticated as the normal user attacker@test.com.

Step 2: Result: HTTP 200 with the complete application configuration as JSON (Figure 2) — server port and base URL, application domain, chatbot model configuration, the Google OAuth client ID, the product catalogue, security-contact details, and challenge configuration.

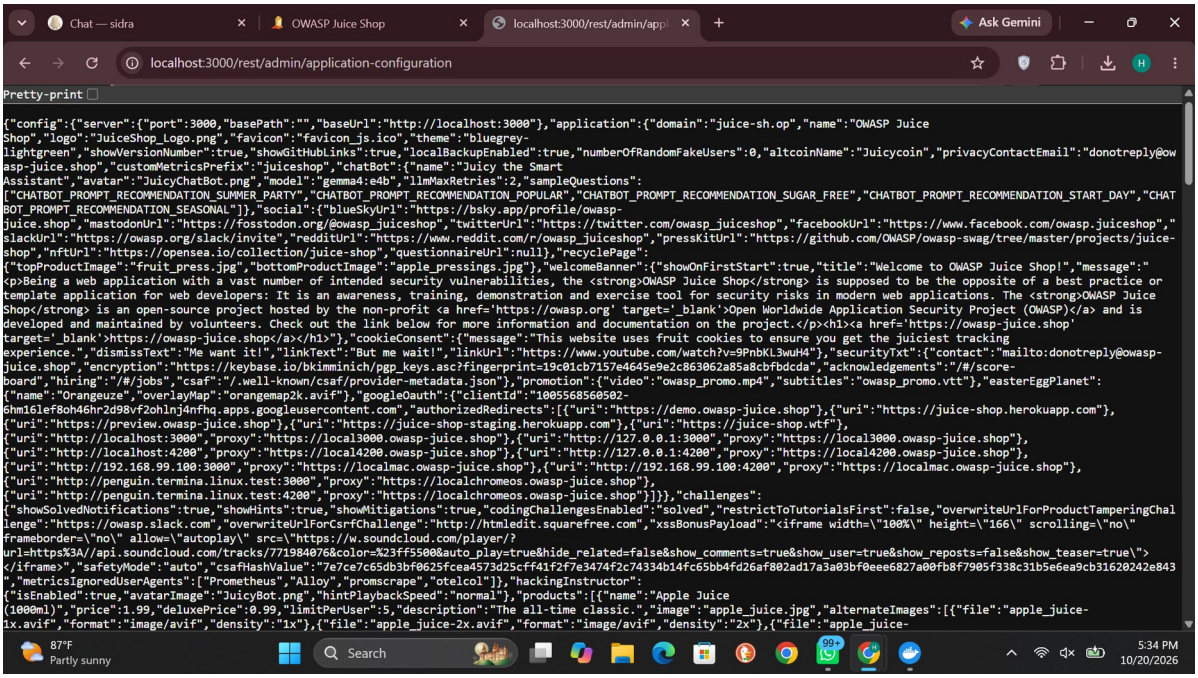


Figure 2 — Full application configuration returned to a normal, non-admin user

Root cause: the endpoint lives under the administrative /rest/admin/ path but performs no server-side authorization check. The 403 on the web page creates a false sense of security: the interface is hidden, but the data behind the admin area is served to anyone who asks. Authorization enforced only in the frontend is not a security control.

Impact

- Disclosure of internal configuration (infrastructure details, third-party client IDs, feature flags) to any registered user.
- The exposed data materially assists further attacks — reconnaissance for Projects 1–2 style follow-ups, targeted phishing, and abuse of disclosed endpoints.
- The same missing-check pattern on a sibling admin endpoint could expose far more sensitive operations.

Remediation

- **Primary fix —** Enforce authentication and an administrator role check on the server for every /rest/admin/* route, returning 401/403 to unauthorized callers.

```
// VULNERABLE — admin route with no authorization check
app.get('/rest/admin/application-configuration', (req, res) => { /* ... */ });

// FIXED — authentication + role check enforced server-side
app.get('/rest/admin/application-configuration',
        authenticate, requireRole('admin'),
        (req, res) => { /* ... */ });
```

- **Hardening —** Audit the whole route table for endpoints that rely on the UI to hide them, and log rejected authorization attempts for monitoring.

5. Test 2 — Finding 2: Insecure Direct Object Reference (Basket IDOR)

Severity: High

Affected endpoint: GET /rest/basket/:id (basket loaded from the client-held basket id)

CWE: CWE-639 — Authorization Bypass Through User-Controlled Key

Summary. The application identifies the current basket by an identifier (bid) stored in the browser's session storage and supplied by the client on every request. The server returns whichever basket the identifier names, without verifying that the basket belongs to the authenticated user. Changing one digit therefore hands one user another user's basket.

Proof of concept

Step 1: As the victim (incognito window), add 2 × Apple Juice to the basket. In DevTools → Application → Session storage → http://localhost:3000, the victim's basket identifier is bid = 7 (Figure 3).

Step 2: As the attacker (main window), check the same storage: the attacker's own identifier is bid = 6, and the attacker's basket is empty (0 items).

Step 3: In the attacker's session storage, edit the bid value from 6 to 7 and press Enter.

Step 4: Reload the page and open Your Basket. The application loads basket 7: the victim's 2 × Apple Juice (total 3.98) is displayed under the attacker's session, and the "View Basket" challenge is solved (Figures 4 and 5).

Step 5: The basket is fully interactive in the attacker's session — quantities can be changed, items removed, and checkout started against the victim's basket.

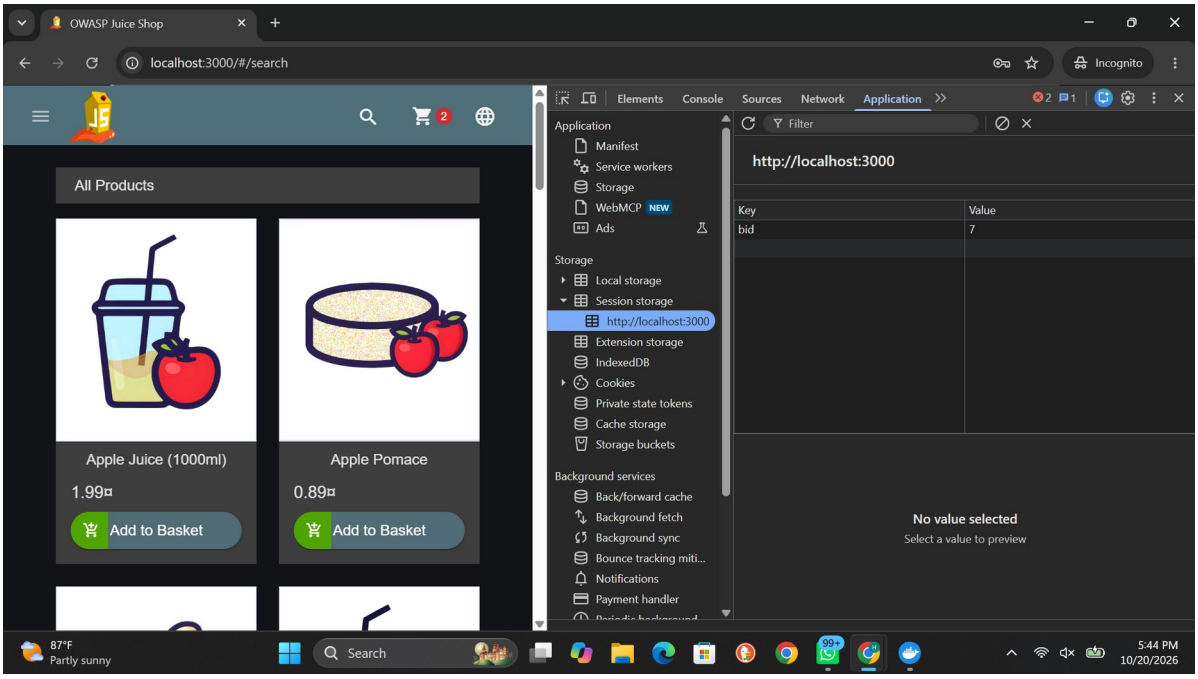


Figure 3 — Victim's session storage: basket identifier bid = 7

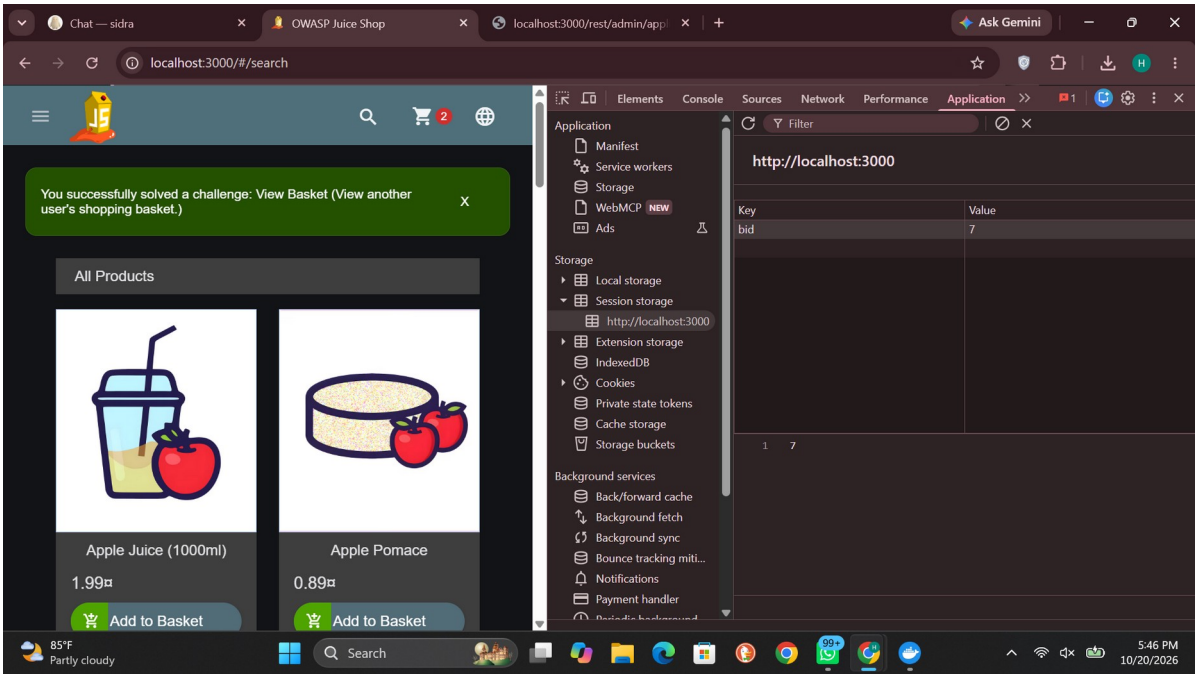


Figure 4 — Attacker's session after tampering: bid changed to 7 and the "View Basket" challenge solved

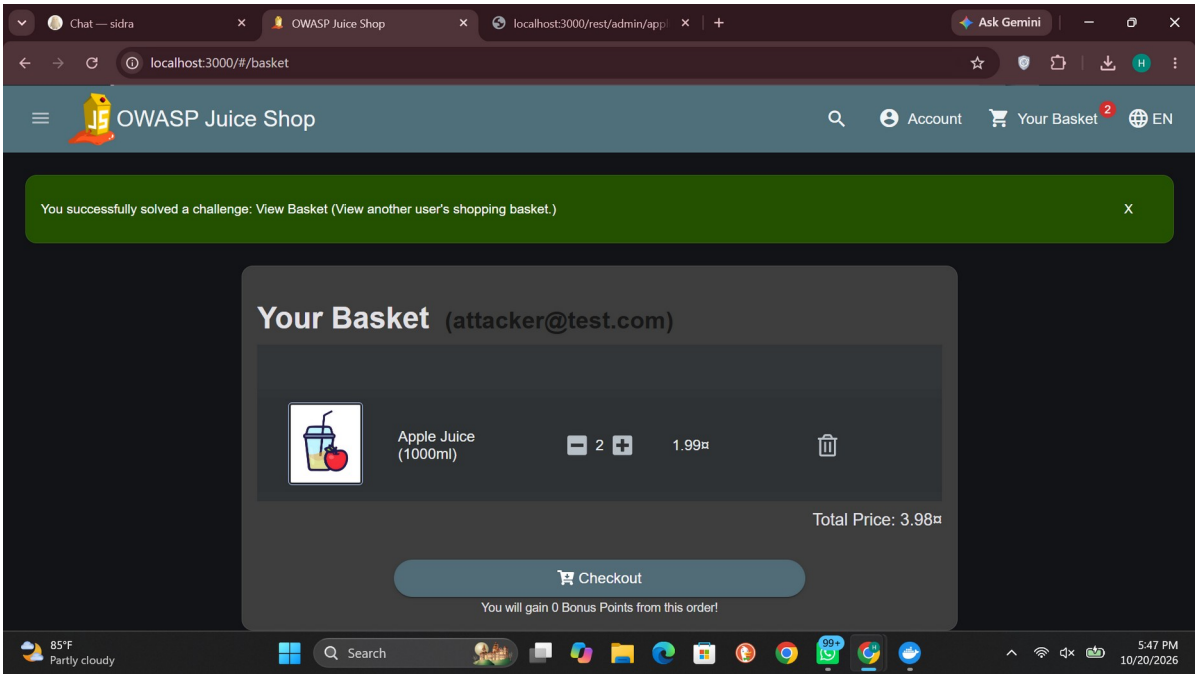


Figure 5 — The victim's basket (2 × Apple Juice, total 3.98) displayed under the attacker's account (attacker@test.com)

Impact

- Confidentiality: any user can read any other user's basket contents — a privacy breach and a source of behavioural data.
- Integrity: the same access allows modifying or deleting the victim's basket and initiating checkout with it.
- Identifiers are small sequential integers, so enumerating every basket in the shop is trivial (6, 7, 8, …).

Remediation

- **Primary fix —** On every request, verify on the server that the requested object belongs to the authenticated user; reject mismatches with 403. Never trust a client-supplied identifier as proof of ownership.

```
// VULNERABLE — trusts the client-supplied basket id
app.get('/rest/basket/:id', (req, res) => {
  Basket.findByPk(req.params.id).then(b => res.json(b));
});

// FIXED — ownership verified server-side on every request
app.get('/rest/basket/:id', authenticate, (req, res) => {
  Basket.findByPk(req.params.id).then(b => {
    if (!b || b.UserId !== req.user.id)
      return res.status(403).json({ error: 'Forbidden' });
    res.json(b);
```

```
  });
});
```

- **Hardening —** Prefer deriving the basket from the session token instead of accepting an id at all; where ids are unavoidable, apply the ownership check centrally (middleware/policy) so no route can forget it.

6. Findings Summary

#	Test / Finding	Location	Severity	Result
1A	Forced browsing — administration page	/#/administration	—	PASS: 403 returned to non-admin user
1B	Finding 1 — admin configuration API exposed	GET /rest/admin/application-configuration	High	FAIL: full config served to any user
2	Finding 2 — basket IDOR	GET /rest/basket/:id (bid tampering)	High	FAIL: victim's basket viewed and editable

7. Annex A — Reusable Broken Access Control Checklist

The tests above generalize into the following checklist, reusable on any web application. "Result" records this audit's outcome on Juice Shop.

#	Check	How to test	Result
1	Route & endpoint inventory	List every page, route and API endpoint (recon map, Project 1) — access control cannot be tested on an unknown surface.	Done (Project 1)
2	Role matrix	For each endpoint, record who should be allowed: anonymous / user / other user / admin. Test against the matrix.	2 findings
3	Forced browsing to	Request admin/hidden	PASS — 403

	admin pages	URLs directly as a normal user (e.g. /#/administration). Expect 401/403.	
4	APIs behind protected pages	Call the API endpoints that serve protected pages — UI protection often hides unprotected APIs.	FAIL — Finding 1
5	Unauthenticated access	Repeat key requests logged out entirely; admin/data endpoints must refuse.	Version/config data served
6	Horizontal IDOR — read	Change object ids (basket, order, profile, file) to another user's and attempt to read.	FAIL — Finding 2
7	Horizontal IDOR — write	With a foreign id, attempt update/delete/checkout to test integrity, not just viewing.	FAIL — basket editable
8	Identifier predictability	Check whether ids are sequential integers (trivially enumerable) or unguessable references.	Sequential (6, 7, …)
9	Vertical escalation	As a normal user, invoke admin-only operations (user management, config, refunds).	Config read succeeded
10	Server-side enforcement	Confirm checks happen on the server for every request — not only in the frontend, and not only on some routes.	Missing on tested APIs
11	Error & logging behaviour	Unauthorized attempts should return generic	403 page OK; API silent

		401/403 and be logged for monitoring.	200

8. Conclusion

Project 3 is complete. The audit produced a result more instructive than a simple pass or fail: the application's visible access control works — the administration page refuses non-admin users with a 403 — while the enforcement behind it does not. An administrative API discloses the full application configuration to any registered user (Finding 1, High), and a one-digit change to a client-held basket identifier hands one user another user's basket, readable and editable (Finding 2, High). Both flaws share one root cause and one lesson: authorization must be checked on the server, on every request, against the authenticated user's identity and ownership — never inferred from what the interface chooses to show. The generalized checklist in Annex A turns these two tests into a repeatable audit for any future target.